

AGENTIC QUANTUM DOT TUNING

Cross-Platform Agentic Orchestrator — System Architecture Blueprint

Version 2.0 • Clean-Slate Redesign • Pre-Publication Working Draft

Synthesising: Original System Analysis • Literature Review (2019–2025) • NotebookLM Architecture Dialogue

0. Design Philosophy & Ground Rules

This blueprint is the single authoritative reference for the redesigned system. It supersedes the hackathon codebase in its entirety. Every architectural decision below is grounded in the peer-reviewed literature reviewed with you and addresses a specific failure mode identified in the system analysis. Nothing is here by accident; nothing from the old system is retained unless explicitly justified.

Five Non-Negotiable Design Principles

1. One executive, clear chain of command. No competing decision-makers. A single planner holds authority; every other module is a service it calls.
2. Physics first. The Constant Interaction Model (CIM) is embedded in the planner's belief state, not bolted on as post-processing.
3. Uncertainty is a first-class citizen. Every prediction carries a confidence interval. The system never acts on a point estimate alone.
4. Hardware agnosticism by contract. The Device Adapter Layer is a strict interface. Swapping GaAs for Si/SiGe requires zero changes above Layer 4.
5. HITL is a genuine gate, not a speed bump. Auto-approval on timeout is removed. HITL triggers are minimal, meaningful, and blocking.

1. Problem Statement — What Are We Actually Building?

1.1 The Physical Problem

Gate-defined semiconductor quantum dots require precise voltage navigation to achieve specific charge configurations. For a double quantum dot targeting the (1,1) state — one electron per dot — an operator must simultaneously tune 3–9 gate voltages in a non-convex landscape where:

- Each voltage change couples nonlinearly to every other gate (capacitive crosstalk)
- The device is physically unique due to charge disorder from fabrication defects
- Every measurement is slow (seconds to minutes per scan)
- Full 2D stability diagrams require 1,000–4,000 individual measurements
- Manual tuning by experts takes hours to days per device
- This bottleneck is the primary obstacle to scaling spin-qubit arrays

1.2 The Automation Gap

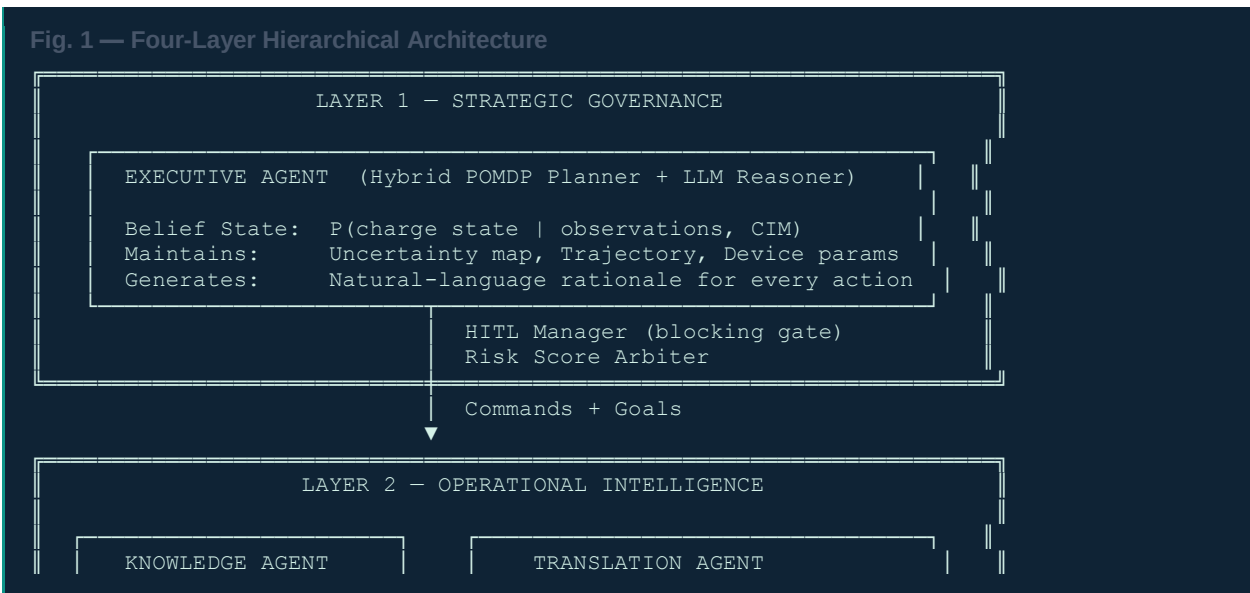
The literature has established that ML can automate coarse tuning and even achieve super-human speed. What it has not addressed:

Gap	Why It Matters	Our Response
No explicit reasoning or planning	Black-box RL policies fail silently and cannot be debugged by physicists	POMDP executive with LLM rationale generation
No multi-fidelity measurement reasoning	Algorithms use fixed scan resolution; line scans are wasted or under-used	Information-theoretic sensing policy with explicit cost/benefit
No interpretability	Experts cannot verify or trust automation decisions	Every action accompanied by a natural-language rationale
No online adaptation	Models trained on one device type fail on another without retraining	Disorder learning + online belief update per device
No hierarchical task decomposition	Single-objective optimisers get stuck in local minima	State-machine executor with autonomous backtracking
Sim-to-real gap is patched, not solved	OOD suppressors hide genuine distribution shifts	Disorder model learns device-specific defect landscape

2. Full System Architecture

2.1 Four-Layer Hierarchy

The system is strictly hierarchical. Information flows downward through commands; observations flow upward through structured reports. No layer may bypass the one above it.

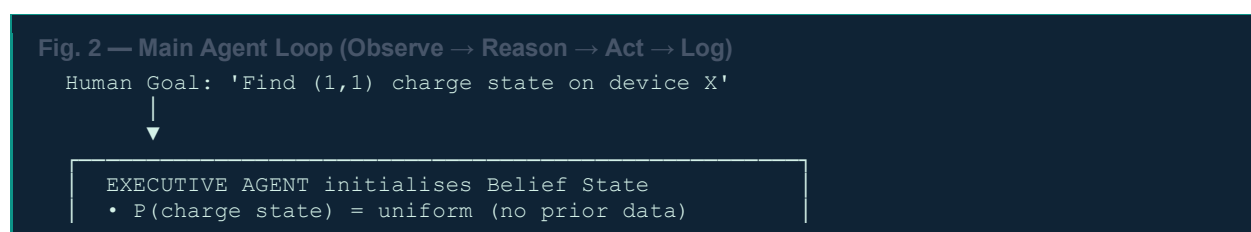




3. Information Flows — Who Talks To Whom, When, About What

3.1 The Main Agent Loop

Each iteration of the main loop follows a strict observe → reason → act → log cycle. The diagram below is the ground truth for implementation.



- Load device config via Knowledge Agent
- Request HITL 'mission start' authorisation → HUMAN approves

MAIN LOOP

1. PLAN NEXT MEASUREMENT

Executive asks: 'What measurement maximises information gain per cost?'

ActiveSensingPolicy computes:
 $I(\text{belief}; \text{measurement}) / \text{cost}(\text{meas})$

Selects: line_scan | coarse_2D |
 local_patch | fine_2D | skip

Measurement plan

2. TRANSLATE & EXECUTE

Translation Agent:
 NL plan → Python call → self-check
 → Device Adapter API → raw current data

Raw conductance array

3. PERCEIVE & FILTER (DQC Gatekeeper)

Score data quality:
 HIGH → pass to Inspection Agent
 MOD → flag + pass with warning
 LOW → STOP: trigger HITL + rebaseline

Quality-gated data

4. CLASSIFY & UPDATE BELIEF

Inspection Agent:
 CNN + physics heuristics → label + conf
 Ensemble → uncertainty estimate

OOD detected?
 YES → trigger DisorderLearner module
 NO → update $P(\text{charge state} | \text{obs})$

NL report → Executive Agent

Updated belief state

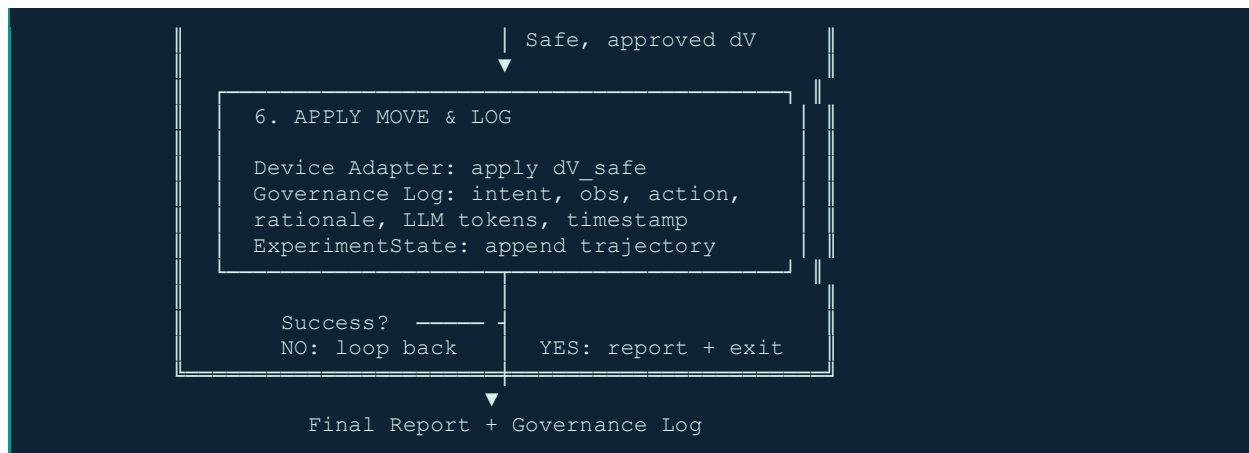
5. DECIDE ACTION

POMDP Planner evaluates:

- Am I confident enough to move?
- What move maximises expected value?
- Is this move safe?

Multi-fidelity BO proposes: dV
 Safety Critic clips: dV → dV_safe
 Risk Score computed: $r \in [0, 1]$

$r > \text{threshold?}$ → HITL gate (blocking)



3.2 Autonomous Backtracking (State Machine)

The tuning process is decomposed into five sequential stages. Each stage has a success criterion and a failure handler. On failure, the agent backtracks to the last stable state and tries a different candidate. After two consecutive backtracks, HITL is triggered.

Fig. 3 — Tuning State Machine with Backtracking

```

Stage 0: BOOTSTRAPPING
Goal: Device responds to gates, charge sensor functional
Success: Current modulation confirmed on ≥1 gate
Failure: Electrical integrity check → HITL (always)
    |
    ▼ pass
Stage 1: COARSE SURVEY
Goal: Locate boundary hypersurface, find any Coulomb peak
Method: Gaussian process maps 20% current threshold surface
        Thompson sampling + ray-tracing along unit vectors
Success: Peak detected in 1D scan → Stage 2
Failure: Prune region, displace origin → retry (max 3)
        3 failures → backtrack to Stage 0
    |
    ▼ pass
Stage 2: CHARGE STATE IDENTIFICATION
Goal: Classify current region (SC / Barrier / SD / DD)
Method: Coarse 2D patch (16×16) → Inspection Agent
        Disorder learning if OOD triggered
Success: label == 'double-dot', confidence ≥ θhi → Stage 3
Failure: label unclear → refine (32×32) → if still unclear backtrack
    |
    ▼ pass
Stage 3: (1,1) NAVIGATION
Goal: Move to (1,1) charge state
Method: Multi-fidelity BO with CIM physics priors
        Update belief at each voltage step
Success: Coulomb anti-crossing visible + label == '(1,1)' → Stage 4
Failure: Confidence drops → re-classify (Stage 2)
    |
    ▼ pass
Stage 4: VERIFICATION
Goal: Confirm (1,1) is stable and reproducible
Method: Repeat scan, check for charge noise, validate anti-crossing
Success: 3 consecutive confirmations → MISSION COMPLETE
Failure: Charge switch noise → flag in governance log + HITL

```

```
BACKTRACKING RULE:  if stage N fails and N.retries ≥ 2:
    • Backtrack to stage N-1 with different candidate point
    • Log backtrack event in governance log
    • If backtrack_count ≥ 2 at same level → TRIGGER HITL
```

4. Human-in-the-Loop (HITL) — Trigger Conditions & Protocol

HITL is a genuine blocking gate. The agent cannot proceed past a HITL trigger until a human explicitly approves, modifies, or rejects the proposed action. Auto-approval on timeout is removed.

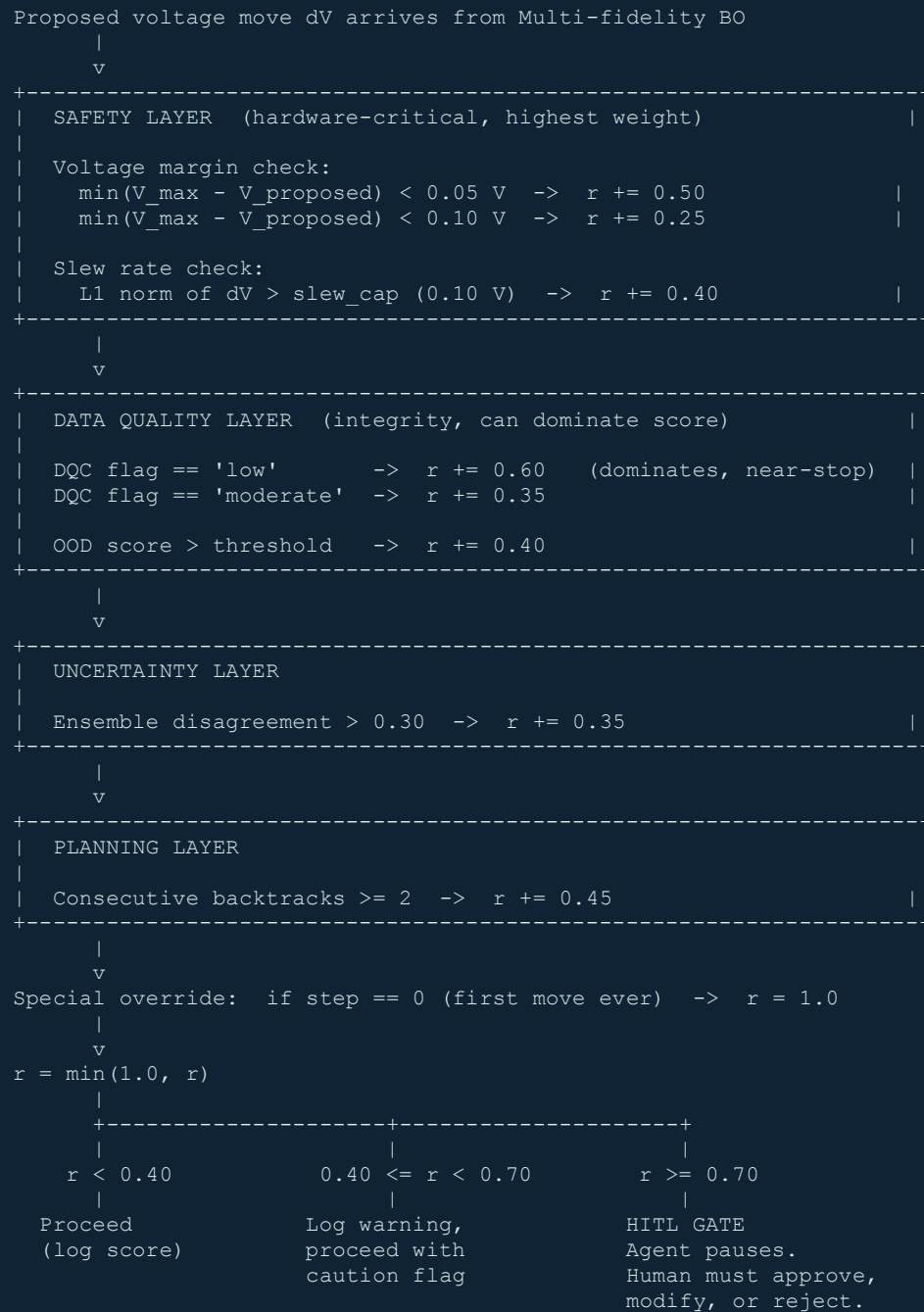
HITL Trigger Conditions (Comprehensive, Device-Agnostic)

#	Trigger	Category	Risk Weight	Agent Action
1	Mission Start — first voltage ever applied to a cold device	Safety	Mandatory	Pause; present plan + full config summary
2	Proximity to voltage limit: $ V_{\text{proposed}} - V_{\text{max}} < M_{\text{min}}$	Safety	0.50	Clip move; present margin + rationale
3	Proposed L1-norm step exceeds slew rate cap	Safety	0.40	Clip move; explain why step was this large
4	DQC Gatekeeper flags: Moderate quality data	Data Integrity	0.35	Trigger targeted recalibration; show flagged scan
5	DQC Gatekeeper flags: Low quality / noise-only data	Data Integrity	Mandatory	STOP; present scan; request manual check
6	OOD score exceeds calibrated threshold	Data Integrity	0.40	Trigger DisorderLearner; present OOD evidence
7	Ensemble classifiers disagree: max disagreement > 0.3	Uncertainty	0.35	Show competing hypotheses; request arbitration
8	Consecutive backtrack count ≥ 2 at same stage	Planning	0.45	Present search tree; request strategy review
9	Cumulative Risk Score exceeds 0.70 in one step	Composite	0.70	Full decision summary; block until approved
10	Escalation from low-cost to high-cost measurement (1D → 2D, configurable)	Efficiency	0.20	Show expected information gain; request approval
11	Stage transition: Coarse → Fine tuning	Planning	0.30	Present stage completion summary; confirm targets
12	Charge switch noise detected (device instability)	Safety	0.50	STOP; log noise signature; request human diagnosis

4.1 Risk Score Formula

The Risk Score is a weighted sum of normalised sub-scores. It is computed before every proposed voltage move and logged regardless of whether HITL is triggered.

Fig. 8 - Risk Score Computation & HITL Trigger Flow



Risk Score Computation

Risk Score r in [0, 1]

```
def compute_risk_score(proposal, state, step) -> float:
    r = 0.0
```

```
# Safety sub-scores
```

```
min_voltage_margin = min(state.voltage_margins.values())
if min_voltage_margin < 0.05: r += 0.50
elif min_voltage_margin < 0.10: r += 0.25

if proposal.ll_norm > CFG['slew_cap']: r += 0.40

# Data quality sub-scores
if state.dqc_flag == 'low': r += 0.60 # dominates
elif state.dqc_flag == 'moderate': r += 0.35

if state.ood_score > CFG['ood_threshold']: r += 0.40

# Uncertainty sub-score
if state.ensemble_disagreement > 0.30: r += 0.35

# Planning sub-score
if state.consecutive_backtracks >= 2: r += 0.45

# First move always triggers HITL regardless of r
if step == 0: return 1.0

return min(1.0, r)

# HITL gate threshold (configurable per deployment)
HITL_THRESHOLD = 0.70
```

5. Module Specifications

5.1 Executive Agent (POMDP + LLM)

The Executive Agent is the single decision-maker. It maintains a belief state over possible charge configurations given all observations to date, using the CIM as its prior. The LLM component (IBM Granite or equivalent) generates the natural-language rationale for every decision.

Attribute	Specification
Planner type	Partially Observable Markov Decision Process (POMDP)
Belief state	$P(n_1, n_2 \mid \text{observations, CIM physics prior})$
Observation model	$P(\text{conductance} \mid n_1, n_2, V_{g1}, V_{g2}; \text{CIM})$
Policy	Expected value maximisation via particle filter + GP
LLM role	High-level strategy selection + rationale text — NOT low-level voltage arithmetic
LLM model	IBM Granite 3-8B-Instruct (swappable via config)
LLM call budget	One call per stage transition + one call per HITL trigger
Rationale format	Structured JSON: {intent, observation_summary, physics_reasoning, proposed_action, expected_outcome}
Online update	Bayesian update of belief at every new observation

Fig. 5 - POMDP Executive Agent: Belief Update & Decision Cycle

```

On initialisation:
Belief State <- Uniform prior  $P(n_1, n_2) = 1/|\text{charge states}|$ 
Physics Prior <- CIM parameters ( $E_c$ ,  $t_c$ ,  $T$ ,  $\alpha$ )
Observation Model <-  $P(G | n_1, n_2, Vg_1, Vg_2)$  via Lorentzian CIM
    |
    v
+-----+
| RECEIVE OBSERVATION  $o_t$  (conductance array from Layer 3) |
+-----+
    |
    v
+-----+
| BAYESIAN BELIEF UPDATE (particle filter) |
| |
| For each particle  $(n_1, n_2)$ : |
|    $\text{weight\_new} \sim P(o_t | n_1, n_2, Vg; \text{CIM}) \times \text{weight\_old}$  |
| Normalise -> updated  $P(n_1, n_2 | o_{1:t})$  |
| If disorder_model fitted: perturb CIM with disorder prior |
+-----+
    |
    v
+-----+
| LLM REASONER (Granite) -- called once per stage transition |
| |
| Input: belief_summary + trajectory + device_config |
| Output: {intent, physics_reasoning, proposed_action, |
|         expected_outcome, rationale_text} |
+-----+
    |
    v
+-----+
| POMDP VALUE COMPUTATION |
| |
| For each candidate action  $a$  in {measure, move, skip}: |
|    $Q(\text{belief}, a) = E[R(s, a)] + \gamma * E[V(\text{belief}')]$  |
|    $R = -\text{measurement\_cost} + \text{success\_bonus}$  |
|    $V = \text{value of next belief state (GP approximation)}$  |
| |
| Select:  $a^* = \text{argmax } Q(\text{belief}, a)$  |
+-----+
    |
    +-----+
    | |
    | a* = measure | a* = propose_move | | |
    | | | | |
    | v | v |
    |> Active Sensing Policy |> Multi-fidelity BO |
    |> Layer 3 Inspection |> Safety Critic -> Layer 4 |

```

5.2 DQC Gatekeeper

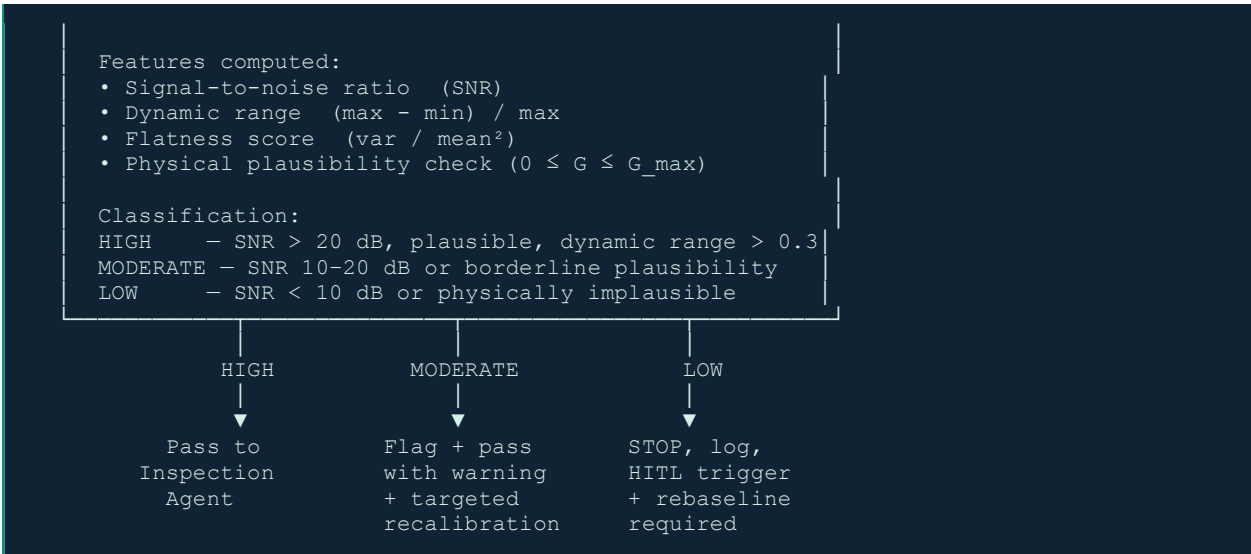
The gatekeeper sits between raw measurements and the Inspection Agent. It is a fast, independent quality classifier that prevents the rest of the system from acting on corrupted data. Unlike the old OOD suppressor, it does not hide problems — it surfaces them.

Fig. 4 — DQC Gatekeeper Logic

```

Raw conductance array (any shape: 1D or 2D)
    |
    v
+-----+
| DQC CLASSIFIER |
+-----+

```



5.3 Inspection Agent (Perception)

The Inspection Agent classifies 2D stability diagrams only. Line scans are used for navigation by the Executive Agent directly; they are not passed to the Inspection Agent. This removes the old bandaid of the line-scan limit.

Component	Method	Output
Log-space preprocessing	$\log_{10}(G + \epsilon)$ normalised to [0,1]	Noise-compressed array
FFT periodicity detector	Peak ratio > 3.5 → single-dot signal	peak_ratio ∈ ℝ
Diagonal edge detector	Laplacian + gradient angle → diagonal_strength	diagonal_strength ∈ [0,1]
CNN classifier (TinyCNN)	3-class: single-dot / double-dot / misc	label + softmax confidence
Physics heuristic validator	CIM-derived threshold rules (device-adaptive, not fixed)	override_flag
Ensemble uncertainty	5-model ensemble: max disagreement = uncertainty	uncertainty ∈ [0,1]
OOD detector	Mahalanobis distance (PCA, calibrated per device online)	ood_score; triggers DisorderLearner if flagged
NL report generator	LLM summarises classification for Executive Agent	Structured JSON report

5.4 Active Sensing Policy (Multi-Fidelity)

Replaces the fixed heuristic thresholds (Ct_low, Ct_high) with an information-theoretic measurement selection policy. Every candidate measurement is scored by expected information gain divided by its cost. The agent picks the best ratio.

Fig. 6 - Active Sensing Policy: Information Gain / Cost Selection
INPUTS: BeliefState (particle filter) + ExperimentState

When the Inspection Agent raises an OOD flag, it now triggers the DisorderLearner rather than suppressing the flag. The DisorderLearner infers device-specific charge disorder from a handful of current traces and injects it into the CIM simulator, closing the sim-to-real gap for this specific device.

Fig. 7 - Disorder Learning Module: Bayesian Disorder Inference Pipeline

```

TRIGGER: Inspection Agent raises OOD flag (Mahalanobis score > threshold)
|
v
+-----+
| COLLECT INPUT DATA (~200 current traces from real device) |
| Each trace: I(Vg1) or I(Vg2) at fixed operating point |
+-----+
|
v
+-----+
| SPARSE GP BAYESIAN INFERENCE (Craig et al. 2024) |
|
| 1. Parameterise disorder as potential landscape V_dis(x,y) |
| 2. Place n=25 inducing points at highest-information locations |
| 3. Prior: smooth GP with physical length scale (gate spacing) |
| 4. Likelihood: P(I_measured | V_dis, CIM) via fast CNN approx. |
| 5. Posterior: V_dis_posterior via variational inference |
+-----+
|
v
disorder_posterior = { mean: V_dis_mean, variance: V_dis_var }
|
v
+-----+
| INJECT INTO SIMULATOR |
| CIMSimulator.inject_disorder( disorder_posterior ) |
| -> Simulator now models THIS specific device |
+-----+
|
v
+-----+
| ONLINE UPDATE (after every new measurement) |
| Sequential Bayes: posterior_new = update(posterior_old, obs_new) |
| Inducing point locations re-optimised every 50 measurements |
+-----+
|
v
Signal Executive Agent: OOD resolved, disorder model fitted
Resume normal agent loop with updated CIM simulator

```

Disorder Learning Module

```

class DisorderLearner:
    """
    Learns device-specific electrostatic disorder.
    Follows Craig et al. (2024): physics-aware Bayesian inference.
    """
    def __init__(self, simulator: CIMSimulator):
        self.simulator = simulator
        self.disorder_estimate = None

    def learn(self, measurements: List[Measurement], n_inducing: int = 25):
        """
        Inputs: ~200 current traces from real device
        Process: Sparse GP on inducing points → posterior over disorder
        Output: Disorder potential map (injected back into simulator)
        """
        disorder_posterior = self._bayesian_infer(measurements, n_inducing)
        self.disorder_estimate = disorder_posterior.mean

```

```

        self.simulator.inject_disorder(disorder_posterior)
        return disorder_posterior

    def update(self, new_measurement: Measurement):
        """Online update: refine disorder estimate with each new observation"""
        self.disorder_estimate = self._online_update(new_measurement)
        self.simulator.inject_disorder(self.disorder_estimate)

```

5.6 Device Adapter Layer

The Device Adapter exposes three primitive operations to the rest of the system. The hardware-specific implementation lives entirely inside each adapter subclass. The Executive Agent never needs to know what is below.

Fig. 9 - Device Adapter Layer: Hardware Agnosticism by Contract



Device Adapter Interface Contract

```

class DeviceAdapter(ABC):
    # Abstract base - never instantiated directly
    @abstractmethod
    def sample_patch(self, v1_range, v2_range, res) -> np.ndarray:
        """Return (res x res) normalised conductance map"""

```

```
@abstractmethod
def line_scan(self, axis, start, stop, steps, fixed) -> np.ndarray:
    """Return (steps,) normalised conductance trace"""

@abstractmethod
def set_voltages(self, voltages: Dict[str, float]) -> None:
    """Apply voltage dict to physical gates or simulator"""

class CIMSimulatorAdapter(DeviceAdapter):    # Used for training + validation
    ...

class SiSiGeHardwareAdapter(DeviceAdapter): # Real hardware: Si/SiGe
    ...

class GaAsHardwareAdapter(DeviceAdapter):    # Real hardware: GaAs/AlGaAs
    ...

# To add a new device type: subclass DeviceAdapter, implement 3 methods.
# Zero changes required above Layer 4.
```

5.7 Safety Critic

Consolidation of the old six overlapping safety checks into one coherent module. The Safety Critic is the last gate before any voltage is sent to hardware. It cannot be bypassed by any agent.

Check	Constraint	Action on Violation
Absolute voltage bounds	$V_{\min} \leq V_{\text{proposed}} \leq V_{\max}$ (device-specific)	Clip to bound; log violation
Slew rate (L1 norm)	$\sum \Delta V_i \leq I1_{\max} = 0.10 \text{ V per step}$	Scale ΔV uniformly to $I1_{\max}$
Per-axis maximum	Removed (redundant with L1 for 2D case)	—
Voltage margin	Integrated into Risk Score (trigger 4.1)	HITL if margin < $M_{\min}$
Physical plausibility	Proposed V must be reachable from current V in one step	Reject if step implies discontinuity

6. Experiment State — Single Source of Truth

The old system scattered state across _MEAS dicts, separate BO history lists, governance JSONL files, and Granite prompt logs. Everything is now centralised in a single ExperimentState object that all modules read from and write to.

```
ExperimentState — Centralised state object
@dataclass
class ExperimentState:
    # Identity
    run_id: str
    device_id: str
```

```
device_params: dict # capacitance matrix, noise, lever arm...

# Spatial
current_voltages: Dict[str, float]
trajectory: List[VoltagePoint] # all positions visited
voltage_bounds: Dict[str, Tuple[float, float]]

# Belief
belief: BeliefState # P(charge | obs, CIM)
uncertainty_map: np.ndarray # per-voltage-space uncertainty
disorder_model: Optional[DisorderPosterior]

# Measurements
measurements: Dict[UUID, Measurement]
classifications: Dict[UUID, Classification]
ood_history: List[OODResult]

# Optimisation
bo_history: List[BOPoint] # (voltage, score) pairs for GP
info_gain_log: List[InfoGainRecord]

# Governance
decisions: List[Decision] # full audit trail
hitl_events: List[HITLEvent]
backtrack_log: List[BacktrackEvent]
stage: TuningStage
consecutive_backtracks: int

# Metrics
total_measurements: int
safety_violations: int
time_elapsed_s: float

def current_belief_summary(self) -> str: # -> LLM-readable string
def propose_action(self) -> ActionProposal: # -> calls POMDP planner
def update(self, obs: Observation): # -> Bayesian belief update
```

7. From Old System to New — Change Register

7.1 What Is Removed and Why

Component Removed	Why Removed	Replaced By
Fixed confidence thresholds (Ct_low, Ct_high)	Empirically tuned to simulator; fail on any other device	Active Sensing Policy (information-theoretic, device-adaptive)
OOD suppressor (99.5th percentile calibration)	Designed to hide false positives, masks real sim-real mismatch	DQC Gatekeeper + DisorderLearner (flag and fix, not suppress)
Line-scan limit bandaidd (max 2 line scans)	Symptom fix for classification gap; no information-theoretic basis	Active Sensing Policy — line scan only if info gain justifies cost
6 overlapping safety checks	Redundant; unclear which binding constraint matters	Safety Critic with 3 clean, non-overlapping constraints

Component Removed	Why Removed	Replaced By
Granite as cosmetic validator	API cost for validation theater; heuristic always won	Granite as co-executive: strategy selection + rationale generation
Scattered state (_MEAS dict, BO list, JSONL)	No single source of truth; impossible to audit	ExperimentState dataclass (unified, queryable, logged)
HITL auto-approve on timeout	Defeats the purpose of human oversight	Blocking HITL — agent cannot proceed without explicit approval
Per-axis voltage cap (redundant with L1)	Redundant for 2D case	L1 norm cap only (Safety Critic)
CNN used only for embedding, not classification	Trained a classifier, ignored its output	CNN as primary classifier; heuristics as physics validator

7.2 What Is Added and Why

New Component	Contribution	Literature Basis
POMDP Belief State	Principled uncertainty over charge configuration; physics prior built in	Standard POMDP theory; NIST workshop 2023 recommendation
Active Sensing Policy	Replaces heuristic thresholds with information gain / cost ratio	Lennon et al. 2019 (4× measurement reduction)
Disorder Learning Module	Infers hidden charge disorder from ~200 traces; closes sim-real gap	Craig et al. 2024 (30–40% success rate improvement)
Ensemble Uncertainty Quantification	5-model ensemble; disagreement = uncertainty; prevents overconfident moves	Standard Bayesian deep learning; Zwolak 2023 recommendation
Backtracking State Machine	Decomposes tuning into stages; backtracks on failure; HITL on repeated failure	Schuff et al. 2025 (fully autonomous spin qubit tuning)
Device Adapter Layer	Hardware agnosticism by contract; zero changes above Layer 4 to switch devices	k-agents paper (LeeQ abstraction); FAIR TASC principles
ExperimentState (centralised)	Single source of truth; fully auditable; enables meta-learning	Software engineering best practice; Zwolak 2023 standardisation
DQC Gatekeeper	Prevents hallucination on corrupted data; classifies quality before classification	Zwolak & Taylor 2023 (DQC module pattern)
Meta-Learner (Phase 2)	Fast adaptation to new device in <5 runs; learns to learn across devices	MAML literature; not yet in QD field — novel contribution

8. Implementation Roadmap

Phase 0 — Foundation (Weeks 1–4)

Deliverable: Clean, testable skeleton with no physics yet

- ExperimentState dataclass with all fields (stub implementations)
- DeviceAdapter ABC + CIMSimulatorAdapter (port your existing CIM physics exactly)
- Safety Critic with 3 checks + full unit test suite
- Governance Logger with structured JSON schema
- HITL Manager (blocking, no auto-approve) + mock UI for testing
- CI/CD pipeline: every commit runs safety fuzz test (5,000 iterations)

Phase 1 — Perception (Weeks 5–10)

Deliverable: Inspection Agent passing QFlow benchmark

- DQC Gatekeeper: train quality classifier on QFlow data (good + bad scans)
- TinyCNN: retrain as primary classifier (not embedding-only)
- Physics heuristics: port your FFT + diagonal detector as validator layer
- Ensemble wrapper: 5 CNN instances; disagreement metric
- OOD detector: Mahalanobis distance; calibrated on device, not suppressed
- Benchmark: $\geq 96\%$ accuracy on QFlow held-out test set
- Benchmark: OOD detector flags 100% of out-of-distribution test cases

Phase 2 — Planning (Weeks 11–16)

Deliverable: Full agent loop in simulation; 50% measurement reduction vs dense baseline

- POMDP planner: particle filter belief state + CIM observation model
- Active Sensing Policy: information gain estimation + fidelity cost model
- Multi-fidelity Bayesian Optimisation: GP with physics priors on voltage proposals
- Backtracking state machine: all 5 stages + failure handlers
- Granite integration: call on stage transitions + HITL triggers only
- Full agent loop: connects all modules through ExperimentState
- Benchmark: $\geq 50\%$ measurement reduction vs dense 64×64 baseline
- Benchmark: $\geq 90\%$ success rate on 100 simulated runs

Phase 3 — Sim-to-Real (Weeks 17–22)

Deliverable: Disorder learner validated; agent generalises across device types

- DisorderLearner: Bayesian inference over inducing points (Craig 2024 method)
- Online belief update: integrate disorder estimate into POMDP on OOD trigger
- Cross-device validation: test on QDsim-generated Si, GaAs, Ge-Si device variants
- Hardware adapter: implement one real-hardware adapter (partner with experimental group)
- Sim-to-real test: train on CIM sim, deploy on QDsim, measure performance gap
- Benchmark: $\leq 20\%$ performance drop from sim to QDsim

Phase 4 — Meta-Learning & Publication (Weeks 23–36)

Deliverable: Paper 1 submitted to PRX Quantum / Nature Communications

- MetaLearner: MAML-based fast adaptation (<5 runs to new device)
- Interpretability study: physicists predict agent actions from rationale (target: >70%)
- Full baseline comparison: vs Bayesian Opt (Ares 2020), Deep RL (Moon 2020), Craig 2024
- Ablation study: effect of each new module (DQC, DisorderLearner, ActiveSensing, etc.)
- Hardware validation: real device results (if collaborator access secured)
- Paper 1 draft: 'Agentic Quantum Dot Tuning with Interpretable Physics-Informed Reasoning'
- Target venues: PRX Quantum, Quantum, Nature Communications

9. Evaluation & Benchmarks

9.1 Primary KPIs

Metric	Target	Baseline (to beat)	How Measured
Measurement efficiency	≥50% reduction vs dense scan	Dense 64×64 = 4,096 pts	Median measurements to success
Success rate (simulation)	≥90%	Deep RL: ~75% (Moon 2020)	% runs reaching target in max steps
Success rate (hardware)	≥70% first attempt	Human expert: ~85% but hours	Real device trials (Phase 3+)
Time to target (sim)	≤70 min equivalent	Human expert: 180 min	Wall-clock time at hardware speed
Sim-to-real gap	≤20% performance drop	Current system: untested	Compare sim vs QDsim vs hardware
Interpretability	≥70% expert action prediction	No prior baseline (novel metric)	User study with physicists
Safety violations	Zero	Zero (fuzz tested)	5,000 fuzz iterations per release
HITL quality	≥90% human approvals are valid decisions	No prior metric	Retrospective analysis of HITL logs

9.2 Ablation Study Design

To claim each module contributes, run ablation experiments removing one module at a time:

Ablation	Module Removed	Expected Effect
A1	Active Sensing Policy → fixed Ct_low / Ct_high	↑ measurements, ↓ efficiency

Ablation	Module Removed	Expected Effect
A2	DisorderLearner → OOD suppressor (old behaviour)	↓ sim-to-real success rate
A3	DQC Gatekeeper → no quality filter	↑ false classifications, ↑ wasted moves
A4	Ensemble UQ → single CNN	↑ overconfident moves, ↑ safety margins hit
A5	POMDP belief state → reactive policy (no memory)	↑ steps, ↓ success on complex devices
A6	Backtracking → no recovery (old linear flow)	↑ failure rate on multi-modal landscapes
A7	Granite rationale → silent agent	↓ interpretability score; no efficiency change

10. Community Engagement & Publication Strategy

10.1 Datasets & Simulators to Use

Resource	Type	Use In This Work
QFlow-lite (NIST)	Real experimental data (10k+ diagrams, 4 classes)	Benchmark perception layer (Phase 1)
QDsim (Greplova 2025)	Open-source simulator (up to 100+ dots, realistic noise)	Sim-to-real validation (Phase 3)
QDarts (Krzywda 2025)	Array transition simulator	Large-array generalisation test (Phase 4)
NIST workshop dataset (2024)	Community benchmark dataset	Standardised comparison with other groups

10.2 Target Collaborators

- Natalia Ares (Oxford) — has measurement infrastructure; likely to test automation
- Eliska Greplova (Delft/QuTech) — QDsim maintainer; natural validation partner
- Justyna Zwolak / Jacob Taylor (NIST) — community coordinators; FAIR TASC alignment

10.3 Publication Plan

Paper	Core Claim	Target Venue	Phase
Paper 1	First LLM agent for QD tuning; interpretable reasoning; multi-fidelity sensing	PRX Quantum / Nature Comms	Phase 4
Paper 2 (optional)	Multi-fidelity active learning: 50% measurement reduction; device-agnostic	ML: Sci. & Tech. / Phys. Rev. Applied	Phase 4
Paper 3 (optional)	Meta-learning for fast adaptation: <5 shots to new device	ICML / NeurIPS workshop	Phase 4+

11. Open Questions for Your Review

The following decisions require your input before implementation begins. They are not blockers for Phase 0, but must be resolved before Phase 2.

Your Input Needed

Q1. Target charge state: This blueprint focuses on the (1,1) double-dot state. Do you want the framework to generalise to arbitrary (n, m) states, or is (1,1) the fixed target for Paper 1?

Q2. Granite vs. open LLM: IBM Granite has watsonx governance integration (useful for logging). If you have access, keep it. If not, any instruction-following model works — the architecture is LLM-agnostic.

Q3. Hardware partner: Phases 3–4 are significantly stronger with real hardware validation. Do you have an existing collaborator, or should we identify one via the NIST community?

Q4. Scope of Phase 1 Paper: The blueprint targets Paper 1 at the full agentic system. An alternative is to publish the DQC + Inspection Agent alone (lower bar, faster). Which timeline suits you?

Q5. Meta-learning priority: MAML-based adaptation is a strong novel contribution but adds 4–6 weeks. Is it in scope for Paper 1 or deferred to Paper 3?